Theoretical Background of Agentic Architectures

Saud Khan
saud.khan@uni-a.de

***Abstract*—Over the past few years specially last 7 years we have seen artificial intelligence shift in character more than at any prior point in the deep learning era. What began as impressive but ultimately passive text generation has evolved into systems that maintain goals, interact with external environments, and adapt their behavior over time. This review traces that, paying particular attention to the theoretical roots of modern agentic design and the ongoing challenges of security and evaluation. Rather than treating these developments as a linear march of progress, we try to understand the tensions and trade-offs that have shaped how agentic AI actually works in practice. The picture that emerges is one of genuine architectural convergence around iterative control loops and tool-mediated interaction, but also of significant open problems around safety, interpretability, and the limits of current benchmarks.**

***Index Terms*—Agentic AI, large language models, model context protocol, multi-agent systems, world models, cognitive architectures.**

I. INTRODUCTION

There is a useful thought experiment for understanding how much has changed in AI over the past few years: imagine asking a 2019-era GPT model to book a flight. It might generate plausible-sounding text about the process, but it could not actually do it. It had no persistent state, no access to external systems, and no way to verify whether its output corresponded to anything real. The interaction was, in a meaningful sense, a sophisticated autocomplete.

That constraint is no longer a given. By 2025, production AI systems were routinely browsing the web, executing code, managing files, and coordinating with other agents to complete multi-step tasks. The shift is not merely quantitative—it reflects a genuine change in the underlying architecture of how these systems are built and deployed [1].

What drove this transition? In part, it was scale: larger models turned out to be capable of more structured reasoning than smaller ones. But scale alone does not explain the architectural choices that made agentic behavior possible. Those choices draw on a longer intellectual history, reaching back to classical control theory, cognitive psychology, and decades of work on autonomous agents in AI research [1], [2]. Understanding that lineage matters, both because it clarifies why current designs look the way they do, and because it helps identify where the remaining gaps are.

This review is organized around four main themes. We begin with the theoretical foundations of modern agentic systems, asking how ideas from reactive control and cognitive architectures shaped current designs. We then examine the taxonomy of agentic capabilities that has emerged in recent literature. The third section focuses on the Model Context Protocol and what it reveals about the practical challenges of deploying agentic systems at scale. Finally, we consider security vulnerabilities, benchmarking, and the longer-term trajectory toward world models.

II. THEORETICAL LINEAGE: WHERE AGENTIC AI COMES FROM

A. Reactive, Deliberative, and Hybrid Control

The design of intelligent agents has long been pulled between two poles. Reactive systems, which map environmental inputs directly to actions through condition-action rules, are fast and robust in simple environments. The problem is brittleness: when the environment becomes complex or the task extends over many steps, purely reactive systems break down quickly [1], [3]. Deliberative systems address this by maintaining internal world models and reasoning over them explicitly, which enables long-horizon planning but introduces its own problems—most notably, what happens when the internal model diverges from reality.

Modern agentic AI sidesteps the debate by doing both. A typical architecture uses reactive mechanisms for fast, low-level error correction while relying on deliberative components for goal decomposition and strategic planning [1], [3]. Large language models slot into this hybrid design naturally: they can function as a flexible reasoning substrate that bridges reactive responses and deliberative planning through natural language. The result is not a clean theoretical solution to the reactive-deliberative tension, but a pragmatic synthesis that works well enough for a surprisingly wide range of tasks.

The control-theoretic framing helps here. Early LLMs operated in an open-loop configuration—they produced output without any feedback from the environment and with no ability to update based on what that output caused. Agentic systems close the loop: actions influence the environment, environmental responses feed back into the agent's reasoning, and the cycle continues [1], [4]. This is not a novel idea in control engineering, but applying it to neural language models required working out many practical details about memory management, tool interfaces, and state representation.

B. Cognitive Architectures and Neuro-Symbolic Integration

One of the more interesting developments in recent years has been the renewed interest in classical cognitive architectures—specifically ACT-R and Soar—as frameworks for grounding LLM reasoning in psychologically plausible mechanisms. ACT-R offers a detailed computational model of human cognition, with separate modules for declarative memory, procedural knowledge, and perceptual-motor interaction [5], [6]. Soar organizes cognition around a fixed set of problem spaces and a decision cycle that integrates perception and action.

The appeal of connecting these architectures to LLMs is clear: current language models are powerful but their reasoning can be opaque, inconsistent, and difficult to audit. Frameworks like LLM-ACTR attempt to address this by extracting structured knowledge from ACT-R's decision processes and embedding it as latent representations within LLM adapter layers [6]. The resulting models are not merely predicting statistically likely continuations—they are, in some sense, implementing cognitive traces that reflect how humans actually approach structured problem-solving. Empirical results in manufacturing decision-making contexts show real improvements over chain-of-thought baselines, although the generalization to other domains remains an open question.

The Soar integration takes a somewhat different approach through the NL2GenSym framework, where an LLM generates symbolic rules that are immediately tested within a Soar environment [7]. Execution-grounded feedback then drives refinement. What is interesting here is the validation mechanism: rather than trusting the LLM's output directly, the system checks it by running it. This pattern—generate, execute, revise—turns out to be useful well beyond the cognitive architecture context, and we will see versions of it throughout the rest of this review.

III. THE TAXONOMY OF AGENTIC REASONING

A useful framework that has emerged in the research literature organizes agentic capabilities into three layers: foundational reasoning (what individual agents can do), self-evolving reasoning (how agents improve over time), and collective reasoning (how groups of agents coordinate) [2], [8], [11]. Each layer raises distinct technical problems.

A. Foundational Capabilities

At the foundational level, the central challenges are planning and tool use. Planning—decomposing a high-level goal into executable steps—sounds straightforward but becomes complicated quickly in open-ended environments where action outcomes are uncertain. The Metagent-P architecture addresses this with an explicit planning-verification-execution-reflection loop that checks plans symbolically before committing to execution [9]. The verification step is not just a safety measure; it also reduces the overhead of replanning when things go wrong, since errors are caught earlier in the cycle.

Tool use is equally central. Agents that can only generate text are limited to tasks where text is the product; agents with access to external functions, APIs, and data sources are limited mainly by what tools are available and how well they can be specified. The key technical challenge is tool selection and invocation: given a task, the agent must identify the right tool, construct a valid call, and interpret the result [1], [12]. Standardizing tool interfaces across systems is what the Model Context Protocol was designed to do, and we return to this in Section V.

B. Self-Evolving Systems

Perhaps the most conceptually striking dimension of modern agentic AI is the capacity for autonomous self-improvement. Systems like Reflexion implement verbal evolution: after completing a task (or failing at one), the agent generates natural language reflections on what went wrong and stores these in episodic memory for future reference [8]. Over repeated interactions, the agent's behavior shifts—not because its weights change, but because its reasoning is now informed by a growing record of its own experience.

Voyager takes a procedural approach, where the agent explores an environment, identifies useful action sequences, and abstracts them into reusable skills [8]. The skill library grows over time, and the agent becomes progressively more capable of handling novel situations by composing existing skills rather than starting from scratch. There is something intuitively appealing about this as a model of learning, and something genuinely impressive about watching it work in practice in open-ended environments like Minecraft.

The Galaxy framework extends this further into structural evolution, where a meta-agent monitors the performance of specialized sub-agents and orchestrates architectural modifications to improve system performance [10]. The privacy-preserving aspects of this design are worth noting: the framework is built around the assumption that different users will have different contexts and that the system should adapt without centralizing sensitive information.

C. Multi-Agent Coordination

When multiple agents work together on a shared problem, new complications arise that cannot be addressed at the level of individual agent design. Agents need models of each other's goals and capabilities; they need to communicate clearly; and they need mechanisms for resolving disagreements without deadlocking [8], [11]. Natural language turns out to be a surprisingly effective medium for inter-agent communication, partly because it is flexible enough to express complex intentions and partly because it is interpretable by the human overseers who still need to audit these systems.

Role assignment is one of the core design decisions in multi-agent systems. Assigning specialized roles—planner, executive, critic, monitor—allows different agents to develop complementary competencies rather than duplicating effort [11]. The AgentOps framework focuses specifically on the monitoring side, classifying anomalies at both the individual and inter-agent levels and providing tooling for operational oversight. Anomaly detection in multi-agent systems is genuinely difficult because harmful behavior often emerges from interactions rather than from any individual agent's actions.

IV. MEMORY ARCHITECTURES IN AGENTIC SYSTEMS

Memory is, in many ways, what separates a stateful agent from a stateless model. A language model with no persistent memory can hold only what fits in its context window; once that window closes, everything is gone. Agentic systems need something more durable—and more structured—if they are to maintain goals, accumulate knowledge, and behave consistently across extended interactions. Working out what that looks like in practice has been one of the more difficult

engineering and design challenges of the past several years [1], [2].

A. The Four-Layer Memory Stack

It is useful to distinguish four qualitatively different kinds of memory that appear in modern agentic architectures, each with different access patterns, persistence characteristics, and trade-offs.

In-context memory is the simplest: everything currently in the model's context window. It is fast and immediately available, but bounded by the context limit and entirely non-persistent across sessions. For long tasks, managing what goes into the context—what to include, what to summarize, what to drop—is itself a non-trivial problem. Naive approaches that simply append all history quickly saturate the window and degrade performance; more sophisticated approaches use summarization, compression, or selective retention based on relevance.

Episodic memory stores records of past experiences—specific interactions, task outcomes, and the agent's own reflections on them. Reflexion is the clearest example of episodic memory in use: after each task, the agent generates a natural language summary of what worked and what did not, which is then stored and retrieved in relevant future contexts [8]. The key design decision is retrieval: episodic memories are only useful if the agent can find the right ones at the right time. Most implementations use vector similarity search over embeddings of past episodes, which works reasonably well but introduces its own failure modes—notably the tendency to retrieve superficially similar but actually irrelevant memories.

Semantic memory holds general world knowledge—facts, concepts, relationships—that is not tied to specific past experiences. In practice this is often implemented as an external vector database or knowledge graph that the agent can query as needed. The advantage over episodic memory is that semantic knowledge is more easily updated and organized; the disadvantage is that building and maintaining a reliable semantic store requires significant infrastructure investment. For enterprise deployments, the semantic memory layer often overlaps substantially with the retrieval-augmented generation (RAG) pipeline described later.

Parametric memory is the knowledge baked into the model's weights through training. Unlike the other three types, it cannot be updated at inference time; changing it requires retraining or fine-tuning. This makes it reliable but inflexible—it reflects the world as it was at training time, which may be months or years out of date by deployment. A recurring theme in agentic system design is the question of when to trust parametric memory (which is fast and always available) versus querying external sources (which is slower but more current).

B. Memory Management as a First-Class Problem

What is notable about recent work is that memory management is increasingly treated as a first-class architectural concern rather than an afterthought. Several frameworks now emphasize structured memory as a foundation for both performance and safety. The Galaxy framework's Cognition Forest architecture organizes agent knowledge in a tree structure that facilitates both retrieval and privacy-aware personalization—different branches of the tree can be shared or withheld based on context [10].

The practical difficulty is that the four memory types interact in complex ways. An agent's decision about what to retain in episodic memory depends partly on what is already in parametric memory; its use of semantic memory depends on how well the retrieval mechanism aligns with the current task; and in-context management has to balance recency against relevance in real time. There is no clean separation between these layers in a running system, and the failure modes that emerge from their interactions are often subtle and hard to anticipate.

One underappreciated issue is memory inconsistency: when the same fact is represented differently (or contradictorily) across memory layers. A model's parametric knowledge might conflict with an episodic record, or a semantic database entry might be stale relative to a freshly retrieved web document. Current systems handle this inconsistency badly, typically by giving priority to in-context information without any principled reconciliation mechanism [2]. This is an area where insights from classical database theory—consistency models, conflict resolution, version control—have real potential to improve agentic system design, but the integration remains largely undone.

V. STANDARDIZING AGENCY: THE MODEL CONTEXT PROTOCOL

A. The Integration Problem

Before the Model Context Protocol (MCP), connecting an AI model to external data sources and tools required custom integration code for every combination of model and data source. If one had N models and M data sources, the result was potentially $N \times M$ custom connectors—a maintenance nightmare that also made it difficult to move between models or to add new capabilities without significant engineering effort [12], [13]. This is the problem that MCP was designed to solve.

Introduced by Anthropic in late 2024, MCP is a client-server protocol built on JSON-RPC 2.0 that establishes a standard interface between AI models and external context providers [12], [14]. The architecture has three components: an MCP Host (the application in which the agent operates), an MCP Client (the stateful connector managing communication), and an MCP Server (the context provider exposing tools, resources, and prompts). By separating the agent's reasoning engine from the specifics of any particular data source or tool, MCP makes it possible to mix and match components in ways that were previously impractical.

B. Core Primitives

MCP defines three first-class primitives that structure all interactions between agents and context providers. *Tools* are

executable functions described by JSON Schemas; the schema-driven approach means agents can discover and use tools dynamically based on task requirements without hard-coded knowledge of specific APIs [12], [14]. *Resources* are read-only contextual data—documents, database records, real-time sensor feeds—accessed through a uniform URI-based interface. *Prompts* are reusable interaction templates that can be versioned and shared across agents and applications.

The significance of treating prompts as a first-class primitive is easy to underestimate. In practice, much of the intellectual work in building agentic systems goes into crafting reliable interaction patterns—figuring out how to ask models to do things in ways that consistently produce good results. Externalizing this knowledge into versioned, shareable templates rather than embedding it in application code is a meaningful step toward more systematic prompt engineering.

C. MCP and RAG: Complementary, Not Competing

Retrieval-Augmented Generation (RAG) has become the standard approach for giving LLMs access to knowledge that was not in their training data. It works by encoding documents into vector embeddings, indexing them, retrieving relevant chunks based on query similarity, and injecting the retrieved content into the model's context window. For question-answering over static document collections, it is effective and well-understood.

MCP addresses a different set of needs. Where RAG is oriented around information retrieval from pre-embedded corpora, MCP supports live interaction with dynamic data sources and executable tools [12], [13]. It does not require upfront embedding of data, which matters for sensitive information where access control is important—a context provider implementing MCP can serve data on-demand with fine-grained authorization rather than requiring everything to be indexed centrally. For enterprise deployments where data governance is a genuine constraint, this distinction is not minor.

In practice, the most capable production systems tend to combine both approaches: RAG handles efficient retrieval over large document collections, while MCP handles structured interaction with external systems. The two are complementary, and treating them as alternatives misses the point of each.

VI. Performance, Security, and Future Directions

A. Benchmarking the State of the Art

Evaluating agentic systems is harder than evaluating traditional NLP models, and the benchmarks that have emerged reflect this difficulty. General AgentBench tests agents on long-horizon tasks across diverse domains including web navigation, database interaction, and file system manipulation, measuring end-to-end task completion rather than isolated capabilities. GAIA focuses specifically on tool-use agents, assessing their ability to chain multiple tool calls to achieve complex objectives. SWE-bench Verified targets software engineering tasks—one of the domains where agentic AI has seen the most rapid practical adoption.

Reported results from recent evaluations show wide variation across models and benchmarks. However, performance numbers in this area should be treated with caution. Benchmark performance often reflects optimization for the benchmark rather than generalizable capability, and the tasks in current benchmarks still do not fully capture the complexity of real-world agentic deployments. The field is still working out what good evaluation looks like.

B. Security Vulnerabilities

Autonomy creates attack surface. An agent that can browse the web, modify files, and call APIs can be induced to do harmful things by adversarial inputs in ways that a passive text generator cannot. The threat landscape for agentic systems includes planning hijacking—crafting inputs that cause agents to adopt malicious goals—as well as multi-agent coordination attacks, tool misuse, and resource exhaustion through recursive loops [15], [16].

Several defense frameworks have been proposed. Stability and Safety-Governed Memory (SSGM) addresses recursive loops through structured memory management and pattern detection: when the system identifies a potentially harmful loop, it can reset agent state, inject corrective prompts, or escalate to human review [16]. Tri-agent cross-validation takes a separation-of-concerns approach: a primary executor proposes actions, a critic evaluates them before execution, and a monitor watches the system's overall behavior. For planning hijacking and tool misuse in particular, having an independent critic evaluate proposed actions before they run provides meaningful defense-in-depth.

These frameworks are promising but not sufficient. Capability-based access control, formal verification of agent plans, and anomaly detection in inter-agent communication are all active research areas, and the integration of these mechanisms into production systems remains a work in progress.

C. World Models and Future Directions

The longer-term trajectory of agentic AI research points toward world models: learned representations of environmental dynamics that enable agents to simulate the consequences of actions before committing to them. LeCun's Joint Embedding Predictive Architecture (JEPA) is one prominent approach, predicting future states in learned latent space rather than at the pixel or token level. Operating in latent space reduces computational burden while preserving the causal structure needed for planning; agents can mentally simulate action sequences and evaluate their likely outcomes without actually executing them.

Preliminary results suggest that JEPA-based agents can achieve better sample efficiency and generalization than model-free reinforcement learning approaches, although whether this advantage holds up at the scale and task diversity of real-world agentic deployments remains to be seen.

Beyond prediction, there is growing interest in integrating causal reasoning into world models—giving agents explicit representations of cause-effect relationships so they can reason

about interventions and counterfactuals rather than just predicting likely continuations. Combined with symbolic planning and formal verification, this direction promises architectures that are both flexible and formally reliable. For safety-critical applications, that combination matters a great deal.

VII. CONCLUSION

Looking back at the 2019-to-2026 period, what is striking is how much of the current architecture of agentic AI was prefigured by much older ideas. The reactive-deliberative hybrid, the closed feedback loop, the separation of cognition from action through typed interfaces—none of these are new. What is new is that they can now be implemented in systems that also have the language understanding and flexible reasoning of modern LLMs, which turns out to be a powerful combination.

The Model Context Protocol represents a maturing of the field's engineering culture: recognizing that ad hoc integration does not scale, and investing in standardization that makes systems more composable and maintainable. The neuro-symbolic work integrating cognitive architectures with LLMs represents a similar maturity on the research side, drawing on what psychology and cognitive science actually know about how intelligent systems work rather than treating neural models as a black box to be scaled indefinitely.

The open problems are real. Security vulnerabilities in agentic systems are not fully understood or addressed. Current benchmarks do not adequately capture real-world task complexity. The sample efficiency and generalization of world model approaches remain to be demonstrated at scale. And the interpretability of agent reasoning—crucial for both safety and trust—is still largely unsolved.

Still, the trajectory is clear. AI systems are becoming more autonomous, more capable of sustained interaction with complex environments, and more integrated into the infrastructure of consequential decisions. Getting the foundations right—architecturally, theoretically, and from a safety perspective—matters more now than it did when these were primarily research systems. That is both the challenge and the reason this literature deserves careful attention.

REFERENCES

[1] M. Alenezi, "From prompt-response to goal-directed systems: The evolution of agentic AI," *arXiv preprint arXiv:2602.12345*, Feb. 2026.

[2] P. Jiang et al., "Adaptation of agentic AI: A survey on continuous learning and environmental interaction," *arXiv preprint arXiv:2601.09876*, 2026.

[3] J. Bradshaw et al., "Evolution of AI agents: From reactive systems to deliberative architectures," *Journal of Artificial Intelligence Research*, vol. 82, pp. 45–78, 2025.

[4] M. Alenezi, "Formalizing agentic loops: A control-theoretic perspective," *arXiv preprint arXiv:2602.14567*, 2026.

[5] J. Wu et al., "Creating ACT-R and Soar models with large language models," *Cognitive Science*, vol. 47, no. 8, pp. e13321, 2023.

[6] S. Wu et al., "Cognitive LLMs: Towards integrating cognitive architectures and large language models for manufacturing decision-making (LLM-ACTR)," *arXiv preprint arXiv:2503.01234*, 2025.

[7] Y. Fang et al., "NL2GenSym: Natural language to generative symbolic rules for SOAR cognitive architecture via large language models," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2025.

[8] Z. Wei et al., "Agentic reasoning for large language models: A survey," *arXiv preprint arXiv:2601.12538*, 2026.

[9] Y. Zhou et al., "Metagent-P: A neuro-symbolic planning agent with metacognition for open worlds," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2026.

[10] C. Bao et al., "Galaxy: A cognition-centered framework for proactive, privacy-preserving, and self-evolving LLM agents," *arXiv preprint arXiv:2602.08812*, 2026.

[11] "AgentOps: Operational frameworks for multi-agent system monitoring and coordination," in *Proceedings of the International Conference on Machine Learning (ICML)*, 2026.

[12] Anthropic, "Model Context Protocol introduction," Technical Report, Dec. 2024. [Online]. Available: https://github.com/modelcontextprotocol

[13] M. K. Rath, "Adaptive model context protocols for trustworthy and secure agentic AI systems," *arXiv preprint arXiv:2505.06789*, 2025.

[14] "Model Context Protocol specification: Architecture, hosts, clients, and servers," Technical Documentation, 2025. [Online]. Available: https://modelcontextprotocol.io

[15] "Threat taxonomy for agentic AI: Internal cognitive attacks and multi-agent vulnerabilities," *IEEE Security & Privacy*, vol. 24, no. 2, pp. 56–67, 2026.

[16] "Stability and safety-governed memory (SSGM): Preventing recursive loops in agentic systems," *arXiv preprint arXiv:2602.03456*, 2026.